Tecnológico de Costa Rica

Escuela de Computación

IC2101 Programación Orientada a Objetos

Actividad Asincrónica 6

Profesor: Luis Javier Chavarría Sánchez

Kimberly Torres González

II Semestre 2026

14/10/26

Parte 1

1. ¿Qué significa `final` en un atributo?

Que el atributo recibe su valor una sola vez y después no se puede cambiar.

2. ¿Cómo se representa en UML?

Con `{readOnly}` al final.

3. ¿Por qué `Piso` se crea dentro de `Edificio`?

Porque es composición: el piso pertenece solo a su edificio, no se comparte y no existe sin él. Por eso `registrarPiso` recibe datos y no un objeto `Piso`.

4. ¿Por qué `buscarPiso` debe ser privado?

Porque devuelve un objeto `Piso`. Si fuera público, alguien de afuera podría obtener y modificar un piso, rompiendo la composición y el encapsulamiento.

5. ¿Qué pasa con los pisos si `e1` apunta a `null`?

El edificio y sus pisos quedan sin referencias, y el recolector de basura los elimina. Los pisos desaparecen con su edificio.

6. Opinión sobre el diagrama de actividad

Estaba bastante complejo y si me dio bastante que pensar, al menos si se refiera al diagrama del apartado 3 en los que otorga el documento carecen de dificultad de comprensión

7. Con sus propias palabras:

Asociación: una clase conoce a otra y la guarda, pero no es su dueña.
Agregación: una clase agrupa objetos que vienen de afuera y pueden compartirse.
Composición: una clase crea sus partes, no las comparte, y estas mueren con ella.
Dependencia: una clase usa a otra temporalmente, sin guardarla.

8. Dos contextos

Contexto 1: Clínica veterinaria

Una clínica registra dueños y mascotas. Cada mascota puede tener un dueño, que puede cambiar si es adoptada. Las vacunas se registran solo a través de la mascota y se eliminan con ella. Las jornadas de vacunación inscriben mascotas ya registradas, y una mascota puede estar en varias jornadas. El veterinario calcula el costo de la consulta según la especie, pero no guarda qué mascotas atendió.

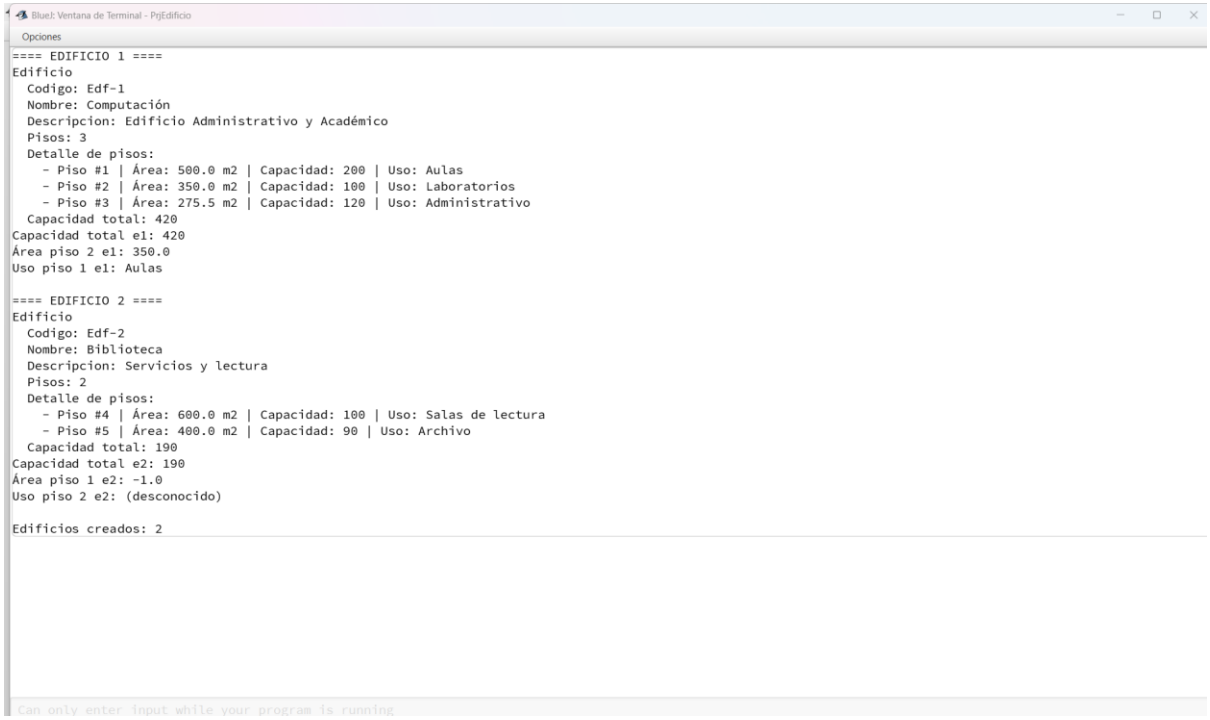
a. `Mascota` con `Dueno`: **asociación**, porque la mascota conoce a un dueño pero existe aparte y puede cambiar.

b. `JornadaVacunacion` con `Mascota`: **agregación**, porque la jornada agrupa mascotas ya registradas que pueden estar en varias jornadas.

c. `Mascota` con `Vacuna`: **composición**, porque las vacunas se crean en la mascota y desaparecen con ella.

d. `Veterinario` con `Mascota`: **dependencia**, porque solo la usa para calcular el costo y no la guarda.

Captura antes de corregir piso:

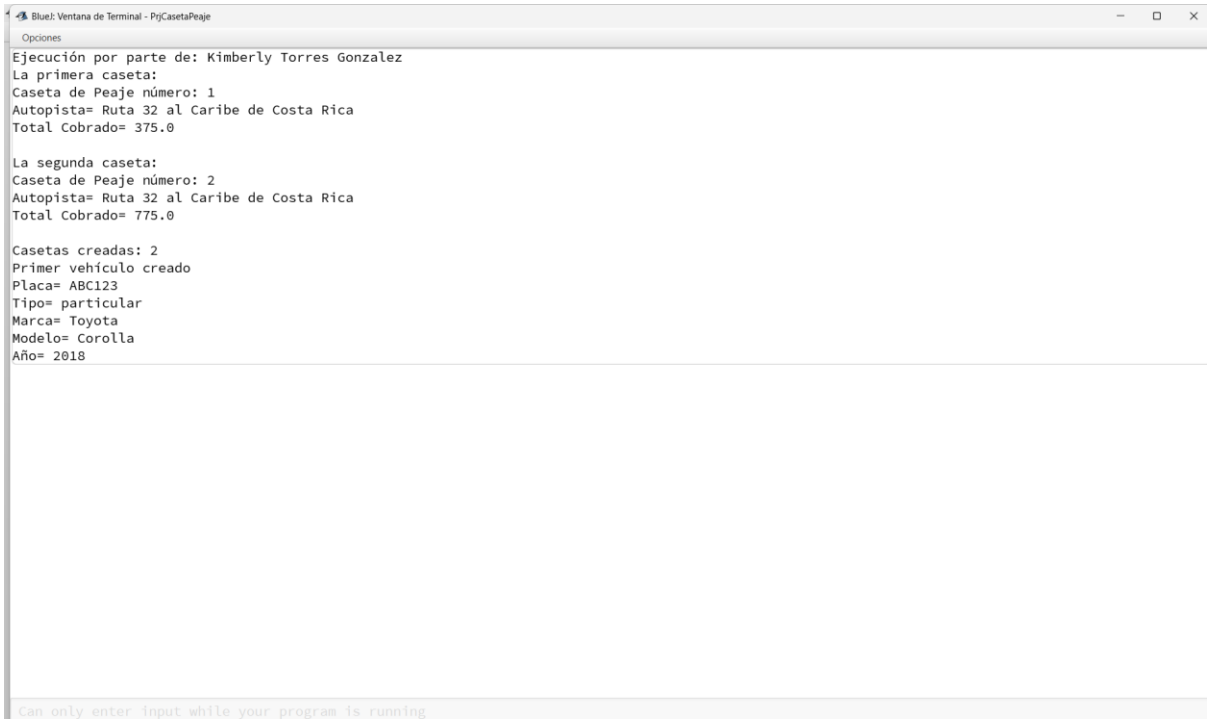


Captura después de corregir Piso:



Caseta Peaje

Captura de la ejecución original:



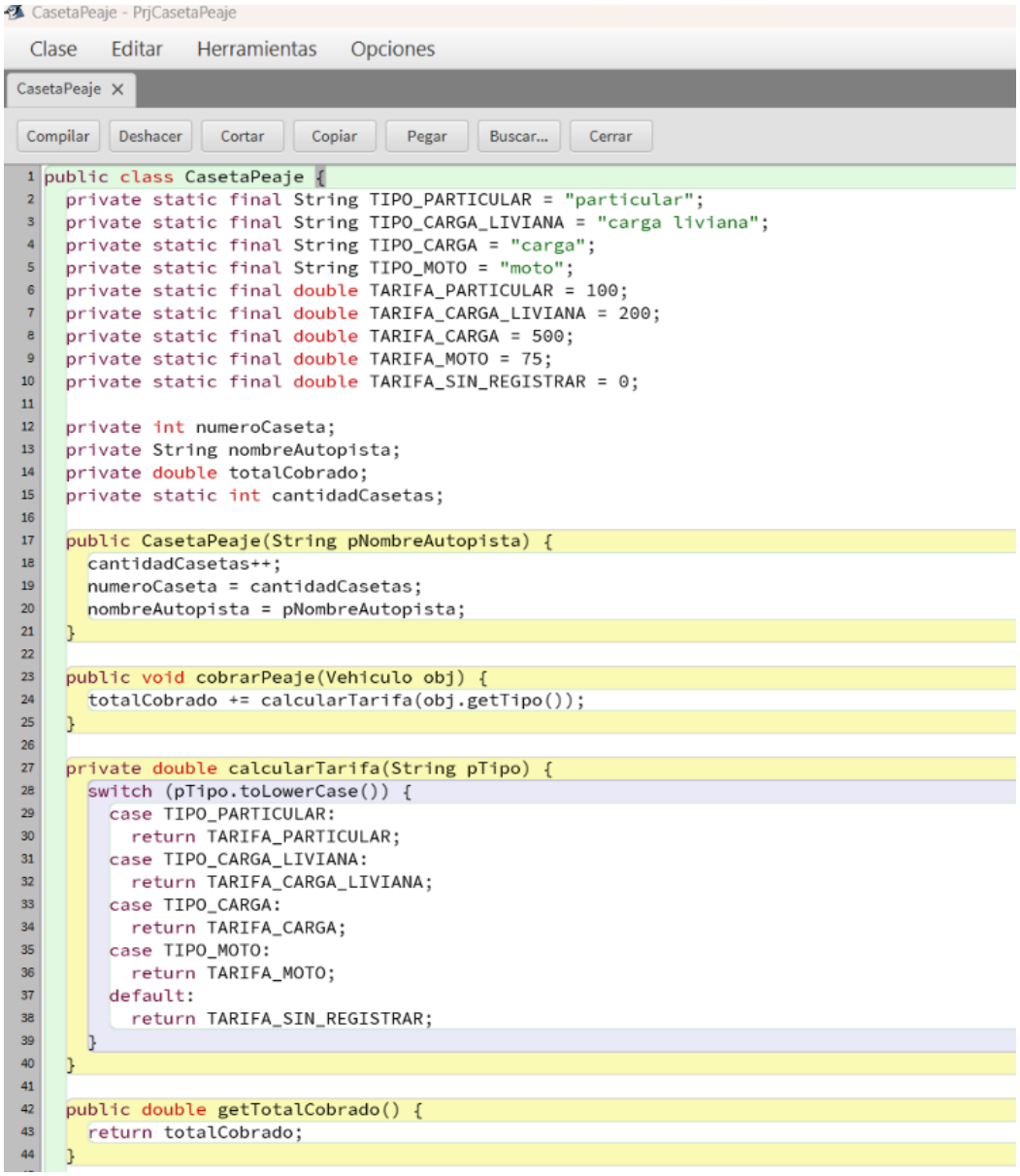
Parte-L

1. Porque `println` llama automáticamente al método `toString()` del objeto.

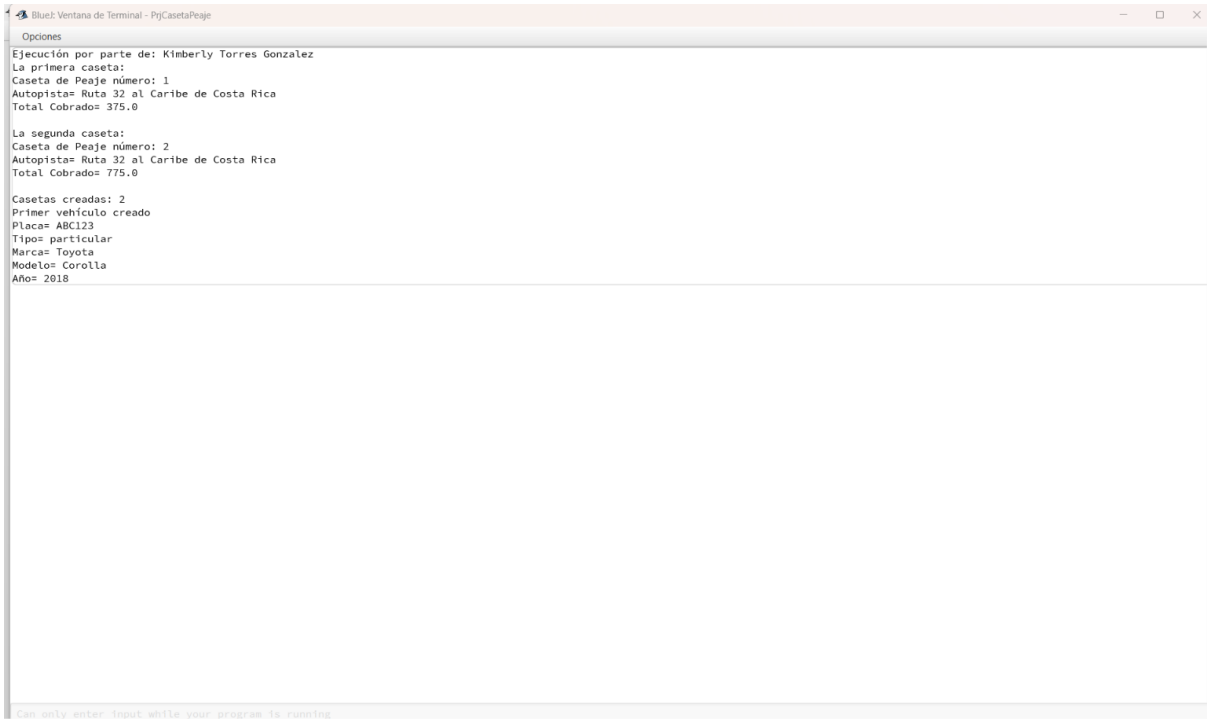
2. Nada: la caseta lo usa para cobrar, pero no lo guarda (dependencia). El vehículo sigue existiendo porque el `main` lo sigue referenciando.

3. Porque `getTipo()` es público, es decir, forma parte de la interfaz de `Vehiculo`, y la caseta recibe el vehículo como parámetro.

Captura implementaciones :



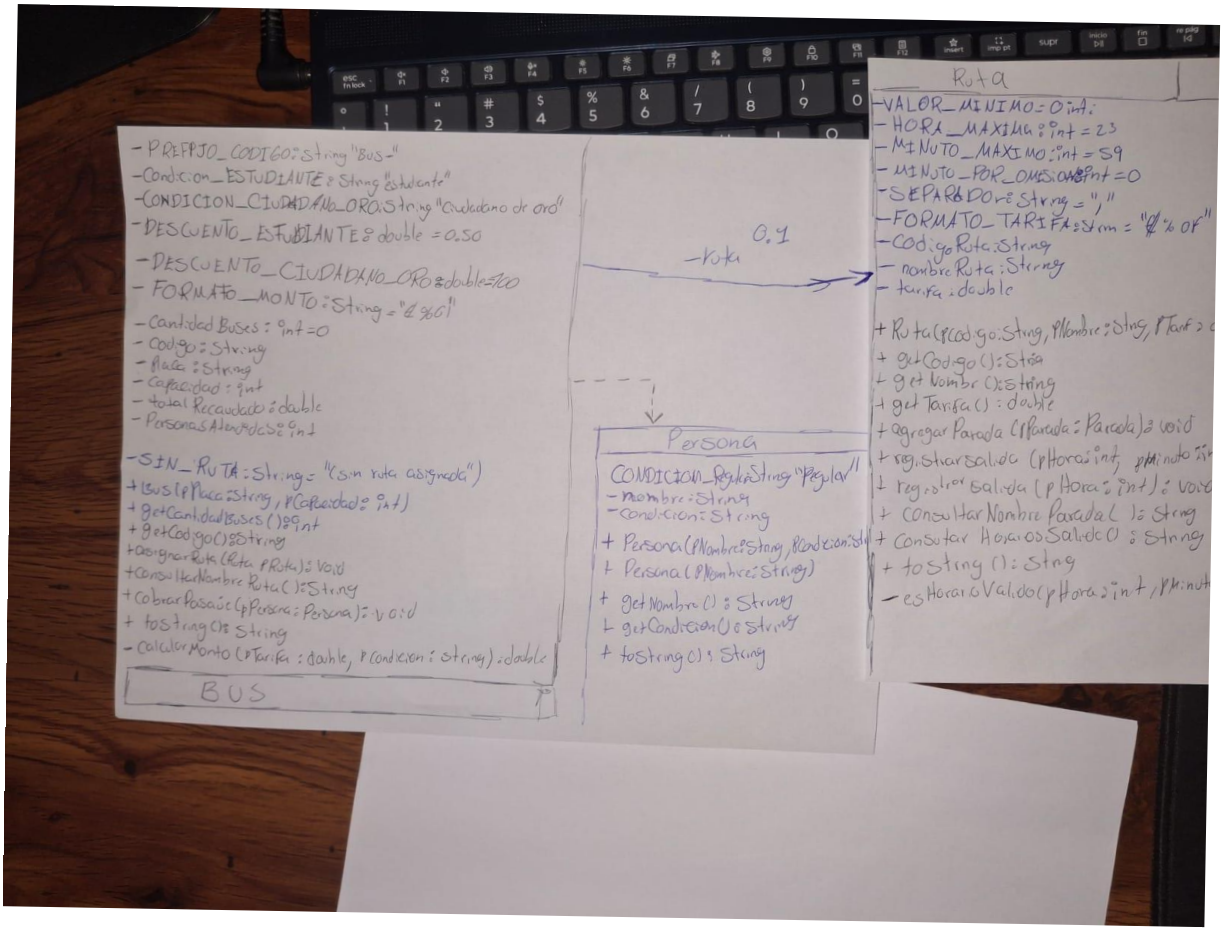
Captura de impresión después de modificación:

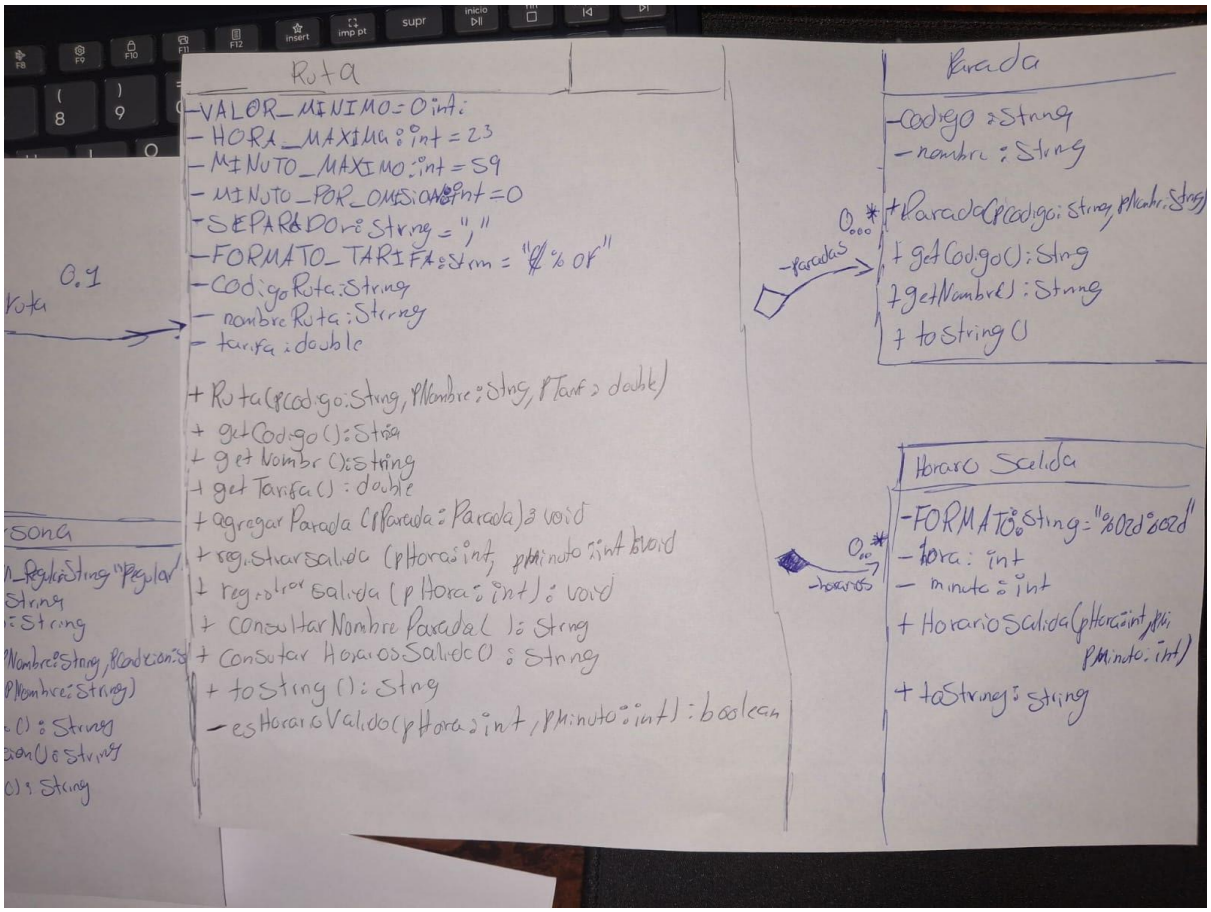


Parte-3

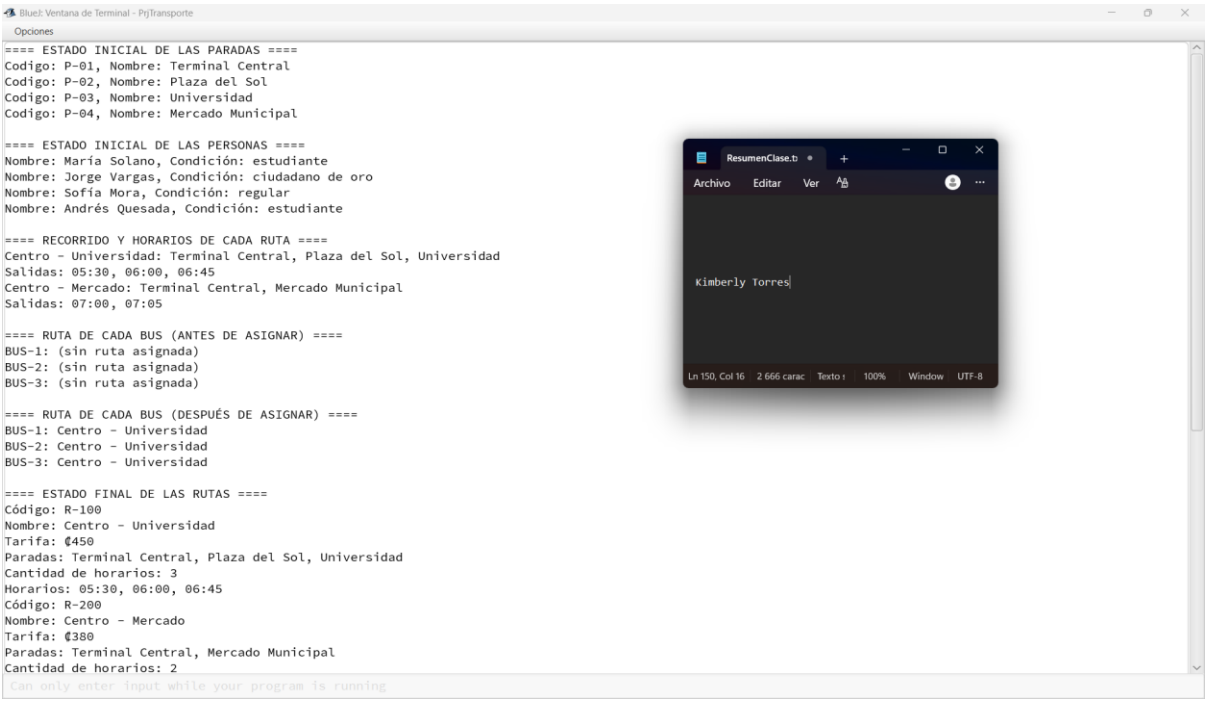


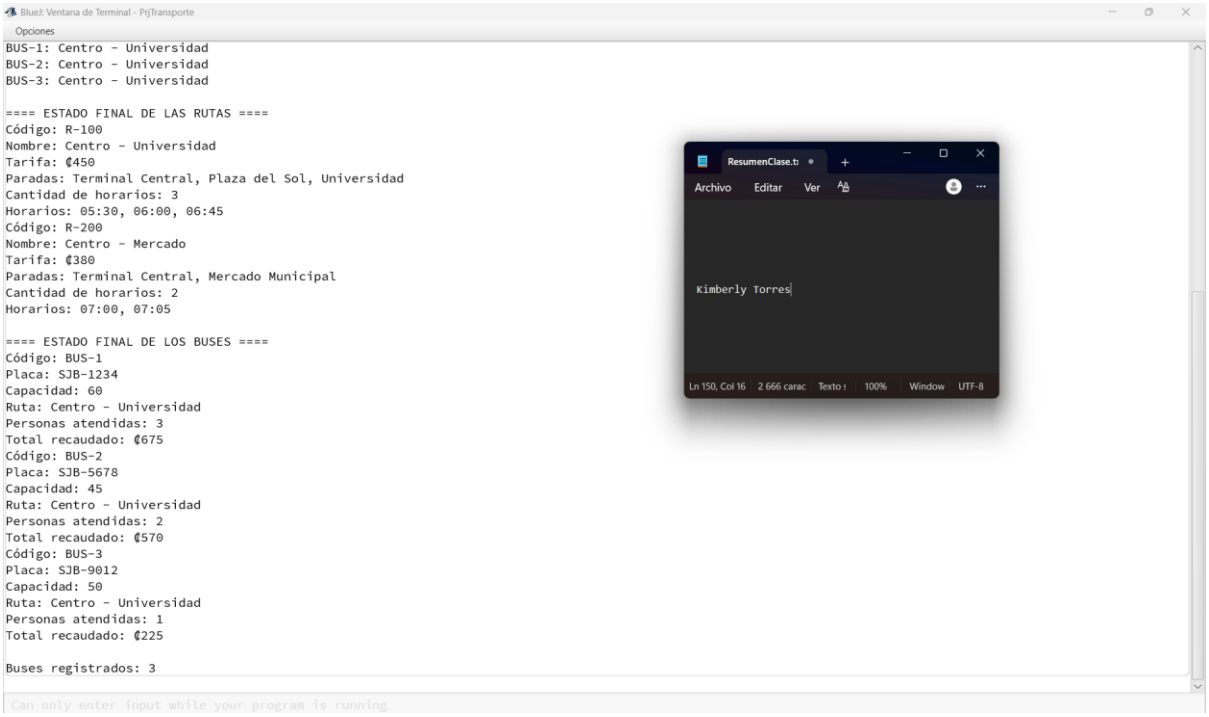
Diagrama de clase con detalles de implementación:


- PREFIJO_CODIGO: String "BUS-"
-Condicion_ESTUDIANTE: String "estudiante"
-CONDICION_CIUDADANO_ORO: String "Ciudadano de oro"
-DESCUENTO_ESTUBIANTE: double = 0.50
-DESCUENTO_CIUDADANO_ORO: double=?.00
- FORMATO_MONTO: String = "¢ %.0f"
-Cantidad Buses: int=0
- Codigo: String
- Placa: String
- Capacidad: int
- total Recaudado: double
- Personas Atendidas: int
-SIN_RUTA: String = "(sin ruta asignada)"
+ Bus(pPlaca: String, pCapacidad: int)
+ getCantidadBuses(): int
+ getCodigo(): String
+ asignarRuta(Ruta pRuta): void
+ consultarNombreRuta(): String
+ cobrarPasaje(pPersona: Persona): void
+ toString(): String
- calcularMonto(pTarifa: double, pCondicion: String): double
BUS
0..1
-Ruta
Persona
CONDICION_REGULAR: String "Regular"
- nombre: String
- condicion: String
+ Persona(pNombre: String, pCondicion: String)
+ Persona(pNombre: String)
+ getNombre(): String
+ getCondicion(): String
+ toString(): String
Ruta
-VALOR_MINIMO: int
- HORA_MAXIMA: int = 23
- MINUTO_MAXIMO: int = 59
- MINUTO_POR_OMISION: int = 0
-SEPARADOR: String = ","
-FORMATO_TARIFA: String = "¢ %.0f"
-CodigoRuta: String
- nombreRuta: String
- tarifa: double
+ Ruta(pCodigo: String, pNombre: String, pTarifa: d
+ getCodigo(): String
+ getNombre(): String
+ getTarifa(): double
+ agregarParada(pParada: Parada): void
+ registrarSalida(pHora: int, pMinuto: i
+ registrarSalida(pHora: int): void
+ consultarNombreParada(): String
+ consultarHorariosSalida(): String
+ toString(): String
- esHorarioValido(pHora: int, pMinut


Ruta
-VALOR_MINIMO: int
- HORA_MAXIMA: int = 23
- MINUTO_MAXIMO: int = 59
- MINUTO_POR_OMISION: int = 0
-SEPARADOR: String = ","
-FORMATO_TARIFA: String = "¢ %.0f"
-CodigoRuta: String
- nombreRuta: String
- tarifa: double
+ Ruta(pCodigo: String, pNombre: String, pTarifa: double)
+ getCodigo(): String
+ getNombre(): String
+ getTarifa(): double
+ agregarParada(pParada: Parada): void
+ registrarSalida(pHora: int, pMinuto: int): void
+ registrarSalida(pHora: int): void
+ consultarNombreParada(): String
+ consultarHorariosSalida(): String
+ toString(): String
- esHorarioValido(pHora: int, pMinuto: int): boolean
0..1
Ruta
Parada
-codigo: String
- nombre: String
+ Parada(pCodigo: String, pNombr
+ getCodigo(): String
+ getNombre(): String
+ toString()
0..*
-paradas
Horario Salida
-FORMATO: String = "%02d:%02d"
- hora: int
- minuto: int
+ HorarioSalida(pHora: int, pMi
pMinuto: int)
+ toString: String
0..*
-horarios

Captura de la ejecución de PrincipalTransporte:





1. ¿Qué relación estableció entre Ruta y Parada?

Agregación. Las paradas se registran aparte, existen sin la ruta y una misma parada puede estar en varias rutas.

2. ¿Cómo representó los horarios? ¿Qué relación tienen con Ruta?

Con la clase `HorarioSalida` (hora y minuto). Es composición: solo la ruta los crea, no se comparten y desaparecen con ella. Por eso nadie de afuera puede obtenerlos.

3. ¿Qué relación estableció entre Bus y Ruta? ¿Por qué esa multiplicidad?

Asociación con multiplicidad `0..1`. Un bus puede no tener ruta (cero) o tener solo una a la vez (uno), y puede cambiarla.

4. ¿Qué relación estableció entre Bus y Persona?

Dependencia. El bus recibe a la persona solo para cobrarle y no la guarda, porque no interesa saber quiénes abordaron.

5. ¿Qué cambiaría si se pidieran los nombres de quienes abordaron un bus?

Pasaría a ser agregación: el bus guardaría una lista de personas, que vienen de afuera y pueden abordar varios buses e.

6. ¿Por qué la navegabilidad va en esa dirección?

Ruta ->Parada: a partir de una parada no interesa conocer sus rutas.
Ruta-> HorarioSalida: el horario no necesita conocer su ruta.
Bus -> Ruta: a partir de una ruta no interesa conocer sus buses.
Bus ->Persona: el bus usa a la persona; ella no necesita conocer al bus.

7. ¿Qué atributos y métodos de clase incorporó?

`cantidadBuses` y `getCantidadBuses()` en `Bus`, además de las constantes. Son de clase porque el contador es uno solo para todos los buses; si fuera de instancia, todos serían BUS-1.

8. ¿Qué constantes incorporó?

Bus: el prefijo "BUS-", "(sin ruta asignada)", las condiciones, los descuentos (0.50 y 1.00) y el formato del monto.
Ruta: los límites de hora y minuto (0, 23, 59), el minuto por omisión (0), el separador y el formato de la tarifa.
HorarioSalida: el formato "%02d:%02d".
Persona: la condición "regular".
Así cada valor se escribe una sola vez.

9. ¿Dónde aplicó sobrecarga y cómo evitó repetir código?

En los constructores de `Persona` y en `registrarSalida` de `Ruta`. La versión corta llama a la completa con el valor por omisión, en lugar de repetir la lógica.

10. ¿Qué métodos privados incorporó y por qué?

`esHorarioValido` (Ruta): es una validación interna de la ruta.
`calcularMonto` (Bus): es un cálculo interno del cobro.

Ambos son detalles internos que no le aportan nada a otros objetos (ocultamiento).

Deciciones de Ambiguedad:

En el diseño de PrjTransporte se tomaron cinco decisiones sobre requerimientos que el contexto no define con precisión.

Tipo de dato para el dinero. El contexto no indica si la tarifa y lo recaudado son enteros o decimales. Se usó double, porque un descuento del 50 % sobre una tarifa impar produce decimales.

Paradas repetidas. El contexto no dice cómo identificar una parada repetida. Se compararon por su código, ya que el código identifica a cada parada.

Validación de horarios. La hora y el minuto se validan en Ruta antes de crear el horario, porque un constructor no puede evitar que el objeto se cree. Así, un horario inválido nunca llega a existir.

Atributos finales. El contexto no indica si el código de un bus puede cambiar. Se declaró final, porque es único y se asigna automáticamente al registrar el bus.